Titanic Survival Prediction Using Deep Learning: An Analysis Report

Author: Thanniru Dharma Nithin

Date of submission : 07-06-2024

COURSERA

Deep Learning and Reinforcement Learning

Objective

The main objective of this analysis is to predict the survival of passengers on the Titanic using a Deep Learning algorithm. Specifically, the focus will be on a neural network model, which is a type of Deep Learning algorithm. This analysis aims to provide insights into which features most significantly influence survival rates, ultimately helping stakeholders, such as historians and researchers, understand the factors that contributed to survival. For businesses, such insights can inform better safety protocols and passenger management strategies in similar contexts.

Data Description

The dataset used in this analysis is the Titanic dataset, which contains information about the passengers aboard the Titanic. The key attributes include:

- **PassengerId**: Unique ID for each passenger
- **Survived**: Survival status (0 = No, 1 = Yes)
- **Pclass**: Ticket class (1 = 1st, 2 = 2nd, 3 = 3rd)
- **Name**: Passenger name
- **Sex**: Gender
- **Age**: Age of the passenger
- **SibSp**: Number of siblings/spouses aboard
- **Parch**: Number of parents/children aboard
- **Ticket**: Ticket number
- **Fare**: Passenger fare
- **Cabin**: Cabin number
- **Embarked**: Port of embarkation (C = Cherbourg, Q = Queenstown, S = Southampton)

The goal of this analysis is to build a model that accurately predicts the 'Survived' attribute based on the other features.

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Load the dataset
df =
pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master
/titanic.csv')
# Display the first few rows of the dataset
print(df.head())
# Summary statistics
print(df.describe())
# Data dictionary
```

```
data_dict = {
 'PassengerId': 'ID of the passenger',
 'Survived': 'Survival (0 = No; 1 = Yes)',
 'Pclass': 'Passenger class (1 = 1st; 2 = 2nd; 3 = 3rd)',
 'Name': 'Name of the passenger',
 'Sex': 'Sex of the passenger',
 'Age': 'Age of the passenger',
 'SibSp': 'Number of siblings/spouses aboard',
 'Parch': 'Number of parents/children aboard',
 'Ticket': 'Ticket number',
 'Fare': 'Fare paid by the passenger',
 'Cabin': 'Cabin number',
 'Embarked': 'Port of embarkation (C = Cherbourg; Q = Queenstown; S =
Southampton)'
}for key, value in data_dict.items():
 print(f'{key}: {value}'))
```

output:

```
PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                  Name     Sex   Age  Sib
Sp  \
0                            Braund, Mr. Owen Harris    male  22.0
1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0
1
2                             Heikkinen, Miss. Laina  female  26.0
0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0
1
4                           Allen, Mr. William Henry    male  35.0
0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000
```

```
            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
PassengerId: ID of the passenger
Survived: Survival (0 = No; 1 = Yes)
Pclass: Passenger class (1 = 1st; 2 = 2nd; 3 = 3rd)
Name: Name of the passenger
Sex: Sex of the passenger
Age: Age of the passenger
SibSp: Number of siblings/spouses aboard
Parch: Number of parents/children aboard
Ticket: Ticket number
Fare: Fare paid by the passenger
Cabin: Cabin number
Embarked: Port of embarkation (C = Cherbourg; Q = Queenstown; S = South
ampton)
```

We will explore these variables using summary statistics and visualizations to understand their distributions and relationships with the survival status

Data Exploration and Cleaning

Initial exploration of the data revealed several preprocessing steps were necessary:

1. **Handling Missing Values**: Rows with missing values in the 'Age' and 'Embarked' columns were dropped.
2. **Encoding Categorical Variables**:
   - The 'Sex' column was converted to numerical values (female: 1, male: 0).
   - The 'Embarked' column was one-hot encoded.
3. **Dropping Irrelevant Columns**: Columns such as 'Name', 'Ticket', 'Cabin', and the original 'Embarked' column were dropped.
4. **Feature Scaling**: StandardScaler was used to standardize the numerical features.

Model Training

Three variations of neural network models were trained:

1. **Model 1: Basic Architecture**
   - Two hidden layers with 64 neurons each, ReLU activation, and 50% dropout.
   - Compiled with Adam optimizer and binary cross-entropy loss.
2. **Model 2: Increased Neurons**
   - Three hidden layers with 128 neurons each, ReLU activation, and 50% dropout.
   - Compiled with Adam optimizer and binary cross-entropy loss.

3. **Model 3: Different Dropout Rates**
    - Two hidden layers with 64 neurons each, ReLU activation, and 30% dropout.
    - Compiled with Adam optimizer and binary cross-entropy loss.

Recommended Model

After evaluating the performance of the three models, the basic architecture (Model 1) was found to be the most balanced in terms of training time and complexity. However, all models experienced NaN values in the loss function, indicating an issue that needs addressing. Despite this, Model 1 is recommended as it is simpler and easier to debug.

Key Findings and Insights

1. **Training Issues**: All models encountered NaN values in the loss function, indicating a potential issue with data preprocessing or model configuration.
2. **Low Accuracy**: The best model achieved a test accuracy of 44.89%, which is below the baseline accuracy of a random guess (50%).

Next Steps

1. **Investigate NaN Values**: Further investigation is needed to identify and rectify the cause of NaN values in the loss function.
2. **Feature Engineering**: Consider additional feature engineering to create more informative features.
3. **Hyperparameter Tuning**: Experiment with different model architectures, learning rates, and batch sizes to improve performance.
4. **Data Augmentation**: Add synthetic data or use cross-validation to enhance model robustness.

By addressing these issues, the model's accuracy and reliability can be significantly improved, leading to more actionable insights for stakeholders.

Model Performance

Code:

```
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
# Load dataset
titanic_data =
pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")

# Data preprocessing
titanic_data.dropna(subset=['Age', 'Embarked'], inplace=True)

# Convert 'Sex' to numerical values (female: 1, male: 0)
titanic_data['Sex'] = titanic_data['Sex'].map({'female': 1, 'male': 0})

# One-hot encode 'Embarked'
encoder = OneHotEncoder(sparse_output=False)
embarked_encoded = encoder.fit_transform(titanic_data[['Embarked']])
embarked_encoded_df = pd.DataFrame(embarked_encoded,
columns=encoder.get_feature_names_out(['Embarked']))
titanic_data = pd.concat([titanic_data, embarked_encoded_df], axis=1)

# Drop columns that are not needed or cannot be easily converted to numeric
titanic_data.drop(columns=['Name', 'Ticket', 'Cabin', 'Embarked'], inplace=True)

# Split features and target
X = titanic_data.drop(columns=['Survived'])
y = titanic_data['Survived']

# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define the model architecture
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(X_train_scaled, y_train, epochs=20, batch_size=32, validation_split=0.2)

# Evaluate the model
loss, accuracy = model.evaluate(X_test_scaled, y_test)
print(f'Test loss: {loss}, Test accuracy: {accuracy}')
```

output:

```
Epoch 1/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 0.5080 -
loss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 2/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5040 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 3/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4877 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 4/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.5030 -
loss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 5/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5063 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 6/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.5178 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 7/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4821 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 8/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5149 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 9/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4868 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 10/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4963 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 11/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5040 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 12/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5159 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 13/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4927 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 14/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.5028 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 15/20
18/18 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.4996 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 16/20
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.4925 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 17/20
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.4886 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 18/20
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4786 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 19/20
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5090 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
Epoch 20/20
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5104 - l
oss: nan - val_accuracy: 0.4855 - val_loss: nan
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.4489 - los
s: nan
Test loss: nan, Test accuracy: 0.4883720874786377
```

In [ ]:

All models encountered NaN values in the loss function during training, which suggests an issue in the preprocessing or model configuration. Despite this, the models were evaluated, and the following observations were made:

- The best model, Model 1, achieved a test accuracy of 44.89%.

Conclusion

The model performance, as indicated by an accuracy of 44.89%, is lower than the baseline accuracy of a random guess (50%). This suggests that the current neural network models are not effective in predicting the survival of Titanic passengers. The presence of NaN values in the loss function further indicates underlying issues that need to be addressed.